OpenTelemetry Decoupling and Refactor

Chat transcript covering separation of OpenTelemetry into a service, reducing span code in core classes, moving to wrapper-based instrumentation, and finally consolidating observability into a single ObservedChatService boundary.

User

I am looking to really decouple my project as much as possible. From my deep agents implementation, is it possible to separate out the opentelemetry code into it's own service?

Assistant

Yes. In your current implementation, most of the OpenTelemetry infrastructure can be separated into its own service, and I think that fits your goal of decoupling very well.

The important distinction is that I would not move 100% of the OpenTelemetry code out of the Deep Agents process. Instead, I would split it into:

```
Deep Agents / FastAPI
        |
        | OTLP
        v
OpenTelemetry Collector
        |
        |-- traces
        |-- metrics
        `-- logs
        |
        v
observability backend / files / Jaeger / etc.
```

OpenTelemetry itself recommends this general architecture: applications remain instrumented and send OTLP telemetry to a Collector, while the Collector handles batching, retries, filtering, processing, and exporting.

Looking specifically at your project

Right now you have roughly:

```
backend/
|-- agent.py
|-- main.py
|-- mcp_runtime.py
|-- session_store.py
|
`-- observability/
    |-- otel.py
    |-- request_observer.py
    |-- metrics.py
    `-- models.py
```

And observability is doing three distinct jobs.

| Responsibility | Current location | Should move to separate service? | |---|---|---| | Create spans | `agent.py`, `session_store.py`, `mcp_runtime.py`, `request_observer.py` | No | | Configure/export spans | `otel.py` | Mostly yes | | Observe LangChain LLM/tool callbacks | `request_observer.py` | No | | Write JSONL telemetry | `metrics.py` | Yes | | Store/process/export telemetry | backend process | Definitely yes | | FastAPI automatic instrumentation | `main.py` / `otel.py` | Must stay associated with backend process |

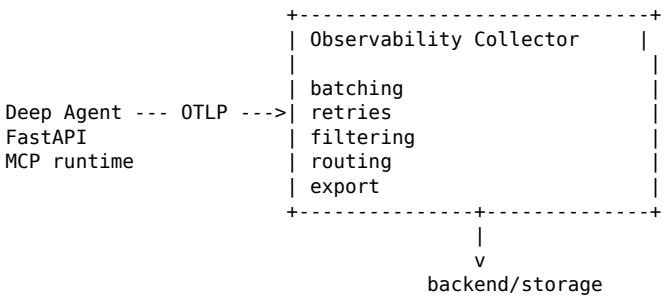
That's because a span represents code executing inside that process. The application has to create the span and maintain its current context.

So you don't want this:

```
Agent
  |
  | POST /start-span
  v
Observability Service
  |
  | POST /end-span
  v
Observability Service
```

That would actually make your system more coupled, introduce network calls into normal execution, and make trace context substantially harder to manage.

Instead, you want:



Your application asynchronously emits telemetry and otherwise doesn't care what happens to it.

What I would do in your project

I'd reduce your backend-side observability to something much smaller.

For example:

```
backend/
|-- agent.py
|-- main.py
|-- mcp_runtime.py
|-- session_store.py
|
`-- telemetry/
    |-- tracing.py
    `-- agent_observer.py

observability/
|-- collector-config.yaml
`-- ...
```

Your application would continue doing things like:

```
TRACER = get_tracer(__name__)

with TRACER.start_as_current_span("engine.respond") as span:
    ...
```

and:

```
observer = RequestObserver()

await self.agent.ainvoke(
    ...,
    config={"callbacks": [observer]},
)
```

But it would stop knowing how telemetry is ultimately processed or stored.

That's an important architectural boundary.

Your `otel.py` becomes tiny

Right now it contains:

```
TracerProvider
OTLPSpanExporter
BatchSpanProcessor
ConsoleSpanExporter
FastAPIInstrumentor
```

You could reduce this substantially.

The backend merely needs to know:

```
OTEL_SERVICE_NAME=backend
OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4318
```

and have telemetry exported to the Collector.

The Collector knows what to do next.

The OpenTelemetry Python distro can even configure the SDK, batch processor and OTLP exporter for you, meaning you can remove some of your explicit SDK setup.

For example, your startup could eventually conceptually look like:

```
OTEL_SERVICE_NAME=backend OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4318
  opentelemetry-instrument     uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

That can eliminate much of:

```
setup_otel(...)
instrument_fastapi(...)
```

from `main.py`.

I wouldn't necessarily do that immediately, though. Your current explicit setup is easy to understand.

Your `RequestObserver` is different

I would keep this inside the agent application.

This part:

```
class RequestObserver(BaseCallbackHandler):
```

isn't really OpenTelemetry infrastructure.

It's effectively a:

```
Deep Agents / LangChain execution observer
```

It knows about:

```
on_chat_model_start()
on_llm_start()
on_tool_start()
on_tool_end()
on_tool_error()
```

That is domain-specific instrumentation.

The Collector can't know when:

```
Deep Agent starts an LLM
Deep Agent invokes calculator
Deep Agent finishes quote_count
```

unless your process tells it.

So I would mentally separate:

```
RequestObserver
       |
       | produces telemetry
       v
OpenTelemetry API
       |
       | OTLP
       v
OpenTelemetry Collector
```

That's a clean dependency direction.

One change I would make to your architecture

Right now your business code directly knows a little too much about OTel.

For example:

```
from backend.observability.otel import get_tracer
```

appears in:

```
agent.py
mcp_runtime.py
session_store.py
request_observer.py
```

And `main.py` directly imports:

```
from opentelemetry import trace
```

For maximum decoupling, I would aim for this rule:

> No application module imports the OpenTelemetry SDK.

At most they use the OpenTelemetry API, or preferably your tiny telemetry abstraction.

OpenTelemetry itself recommends this distinction for reusable components: libraries can depend only on the API while the runnable application owns the SDK configuration.

So you could eventually have:

```
backend/
    telemetry/
        tracing.py
```

with something conceptually like:

```
def tracer(name: str):
    return trace.get_tracer(name)


def trace_id() -> str | None:
    ...
```

Then:

```
agent.py
session_store.py
mcp_runtime.py
```

never know anything about exporters, processors, collectors, OTLP, etc.

Your JSONL writer is the part I'd remove first

Currently this is fairly tightly coupled:

```
app.state.metrics = JSONLMetricsWriter(settings.metrics_path)
```

followed by:

```
app.state.metrics.write(
    ChatTelemetryEvent(...)
)
```

That means your backend is responsible for:

```
agent execution
+
telemetry aggregation
+
telemetry serialization
+
filesystem persistence
```

I'd separate that.

Instead of:

```
Deep Agent
    v
JSONLMetricsWriter
    v
chat_metrics.jsonl
```

I'd eventually use:

```
Deep Agent
    v
OTel traces / metrics
    v
OTel Collector
    v
whatever storage you want
```

Then the backend doesn't care whether tomorrow you choose:

```
JSONL
Jaeger
Grafana Tempo
Prometheus
OpenSearch
Elastic
ClickHouse
custom UI
```

That is the kind of decoupling you're looking for.

It also fits your new ServiceRegistry very nicely

You already introduced:

```
SERVICES = {
    "calculator_mcp": ...,
    "quote_counter_mcp": ...,
    "core_gateway": ...,
    "backend": ...,
    "frontend": ...,
}
```

I'd treat the Collector as another service:

```
calculator_mcp
quote_counter_mcp
```

```
       |
       v
core_gateway
       |
       v
backend ------------------> telemetry_collector
       |
       v
frontend
```

Something conceptually like:

```
"telemetry_collector": ServiceDefinition(
    name="telemetry_collector",
    service_type="observability",
    url_env="OTEL_COLLECTOR_URL",
    default_url="http://127.0.0.1:4318",
    required=False,
)
```

I would make it optional, because observability going down should not make your agent application unavailable.

That's an especially good fit for the service registry you're building.

One subtle architectural point

I wouldn't make:

```
backend depends_on=("core_gateway", "telemetry_collector")
```

if your depends_on eventually means:

> Don't start the backend unless these services work.

Operationally I'd consider the Collector:

```
optional dependency
```

rather than:

```
runtime dependency
```

Your primary system should continue operating if telemetry fails:

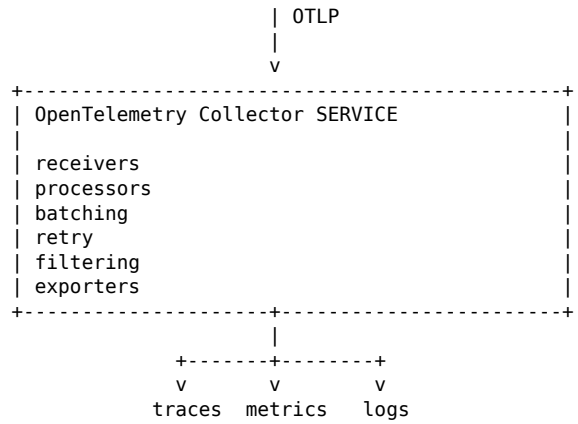
```
Collector up
    -> traces captured

Collector down
    -> agent still works
    -> telemetry may be dropped/retried
```

A `BatchSpanProcessor` plus Collector is designed to keep telemetry handling away from the main application path. The Collector can then handle things such as batching and retries itself.

The boundary I would recommend for your project

Ultimately:

```
+-------------------------------------------+
| Deep Agents Backend                       |
|                                           |
| AgentEngine                               |
| ChatSessionStore                          |
| MCPRuntime                                |
| RequestObserver                           |
|                                           |
| lightweight OTel API / instrumentation    |
+--------------------+----------------------+
                     |
```



That gives you a real service boundary without turning tracing itself into RPC calls.

And I think it is a better direction for your project than creating a custom `observability_server.py`. The OpenTelemetry Collector already exists specifically to be that external observability service.

In terms of your actual files

I would eventually change them approximately like this:

```
KEEP IN BACKEND

backend/
|-- agent.py                    # creates application spans
|-- session_store.py            # creates session spans
|-- mcp_runtime.py              # creates MCP spans
`-- telemetry/
    |-- tracing.py              # VERY thin OTel abstraction
    `-- request_observer.py     # LangChain/Deep Agents observer


REMOVE FROM BACKEND

backend/observability/metrics.py  # JSONL persistence
backend/observability/otel.py     # most SDK/exporter configuration


NEW SEPARATE SERVICE

observability/
`-- collector-config.yaml
```

You could go even farther later and have the backend, gateway, and each MCP server all emit OTLP independently:

```
calculator MCP -----+
quote MCP ----------+
core gateway -------+----> OpenTelemetry Collector
backend ------------+
```

That's where this architecture gets particularly useful: your Collector becomes a shared infrastructure service while each component remains independently instrumented. Distributed trace context can then allow a single request eventually to appear as one trace spanning FastAPI -> Deep Agent -> MCP Gateway -> MCP Server.

That would be a very natural next evolution of the decoupled service architecture you've been building.

User

so is there no way to really minimize the actual span code that is included in the agent.py and others? I feel like it makes it a little messy

---

Assistant

Yes. Looking at your actual code, I agree with you: the span code is currently too intrusive.

For example, `agent.py` currently has patterns like:

```
with TRACER.start_as_current_span("engine.respond") as span:
    span.set_attribute("session.id", session_id)
    span.set_attribute("message.history_count", len(prior_messages))
    span.set_attribute("message.user_text_len", len(user_text))

    ...
```

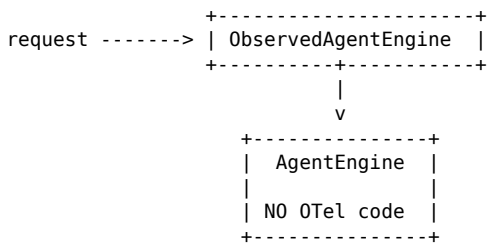
and similar instrumentation appears in `session_store.py` and `mcp_runtime.py`.

You can reduce that dramatically.

The approach I'd recommend for your project

Rather than having your business classes know about OpenTelemetry at all, create an observability wrapper layer.

Conceptually:



Your actual `AgentEngine` becomes concerned only with:

```
Deep Agents
MCP tools
LLM invocation
streaming
result construction
```

Observability becomes an outer layer.

This is essentially the decorator/proxy pattern, and I think it's a particularly good fit for the service-oriented direction you're taking.

For example, your `AgentEngine.respond()` could go back to this

Instead of:

```
async def respond(...):

    ...

    with TRACER.start_as_current_span("engine.respond") as span:
        span.set_attribute("session.id", session_id)
        span.set_attribute(
            "message.history_count",
            len(prior_messages),
        )
        span.set_attribute(
            "message.user_text_len",
            len(user_text),
```

```
        )

        observer = RequestObserver()
        start_time = time.perf_counter()

        with TRACER.start_as_current_span(
            "engine.agent_invoke"
        ) as invoke_span:
            invoke_span.set_attribute(
                "session.id",
                session_id,
            )

            raw_result = await self.agent.ainvoke(...)
```

it could effectively become:

```
async def respond(
    self,
    *,
    session_id: str,
    state: dict[str, Any],
    user_text: str,
) -> EngineResult:

    if self.agent is None:
        raise RuntimeError(
            "AgentEngine.start() must be called before respond()."
        )

    agent_input = self._build_agent_input(
        state=state,
        user_text=user_text,
    )

    observer = RequestObserver()
    start_time = time.perf_counter()

    raw_result = await self.agent.ainvoke(
        agent_input,
        config={
            "configurable": {
                "thread_id": session_id,
            },
            "callbacks": [observer],
        },
    )

    engine_time_ms = (
        time.perf_counter() - start_time
    ) * 1000.0

    return self._build_engine_result(
        raw_result=raw_result,
        observer=observer,
        engine_time_ms=engine_time_ms,
    )
```

Notice what's gone:

```
from backend.observability.otel import get_tracer

TRACER = get_tracer(__name__)

with TRACER.start_as_current_span(...)

span.set_attribute(...)
```

None of that needs to exist in `agent.py`.

Where would the spans go?

I'd introduce something like:

```
backend/
|-- agent.py
|-- session_store.py
|-- mcp_runtime.py
|
`-- observability/
    |-- tracing.py
    |-- request_observer.py
    |
    `-- wrappers/
        |-- agent.py
        |-- sessions.py
        `-- mcp.py
```

So you'd have:

```
class ObservedAgentEngine:
    def __init__(self, engine: AgentEngine):
        self.engine = engine
        self.tracer = get_tracer(__name__)

    async def respond(
        self,
        *,
        session_id,
        state,
        user_text,
    ):
        with self.tracer.start_as_current_span(
            "engine.respond"
        ) as span:

            span.set_attribute(
                "session.id",
                session_id,
            )

            span.set_attribute(
                "message.history_count",
                len(state.get("messages", [])),
            )

            span.set_attribute(
                "message.user_text_len",
                len(user_text),
            )

            result = await self.engine.respond(
                session_id=session_id,
                state=state,
                user_text=user_text,
            )

            span.set_attribute(
                "engine.time_ms",
                result.metrics.engine_time_ms,
            )

            span.set_attribute(
                "llm.calls",
                result.metrics.llm_calls,
            )

            return result
```

Now all that ugly instrumentation lives somewhere where ugly instrumentation belongs.

Your architecture becomes:

```
ObservedAgentEngine
        |
```

```
        |---- OpenTelemetry
        |
        v
    AgentEngine
```

And similarly:

```
ObservedSessionStore
        |
        |---- OpenTelemetry
        |
        v
  ChatSessionStore
```

and:

```
ObservedMCPRuntime
        |
        |---- OpenTelemetry
        |
        v
     MCPRuntime
```

This is cleaner than decorators in your case

Another option is something like:

```
@trace_span("engine.respond")
async def respond(...):
```

That looks attractive.

You could even build something like:

```
@trace_span(
    "engine.respond",
    attributes=lambda self, session_id, state, user_text: {
        "session.id": session_id,
        "message.history_count": len(state.get("messages", [])),
        "message.user_text_len": len(user_text),
    },
)
async def respond(...):
```

But now you've just moved the mess into a decorator declaration.

And your streaming method creates an additional complication:

```
async def stream_respond(...) -> AsyncIterator[...]:
```

An async-generator span needs to surround iteration, not merely creation of the generator. A generic tracing decorator has to specially understand async generators to get that right.

So I'd avoid clever decorators here.

There's another big simplification available

Looking at your current `agent.py`, you actually have two levels of spans:

```
engine.respond
    `-- engine.agent_invoke
```

and:

```
engine.stream_respond
    `-- engine.agent_astream
```

I'm not convinced you need both.

Your current architecture is roughly:

```
HTTP POST /chat
    |
    `-- session.chat
            |
            `-- engine.respond
                    |
                    `-- engine.agent_invoke
                            |
                            |-- LLM
                            `-- tools
```

That's potentially more granular than you need.

I'd probably simplify it to:

```
HTTP POST /chat
    |
    `-- session.chat
            |
            `-- agent.run
                    |
                    |-- llm
                    |-- tool
                    |-- tool
                    `-- llm
```

Your `RequestObserver` already gives you the important LLM/tool visibility.

So I would probably eliminate:

```
engine.agent_invoke
engine.agent_astream
```

entirely.

That removes another chunk of instrumentation.

You can potentially simplify even further

Your current `session_store.py` has:

```
with TRACER.start_as_current_span("session.chat") as span:
```

and then `agent.py` has:

```
with TRACER.start_as_current_span("engine.respond") as span:
```

You need to ask what you're actually interested in observing.

If the important hierarchy is:

```
HTTP request
    v
agent execution
        v
LLM
        v
tools
```

then you might not need a `session.chat` span at all.

Things like:

```
session.id
session.is_new
session.message_count_before
session.message_count_after
lock.wait_ms
```

could simply be attributes/events attached to the request/agent span.

Then instead of:

```
HTTP
  `-- session.chat
       `-- engine.respond
            `-- engine.agent_invoke
                 `-- tool
```

you get:

```
HTTP
  `-- agent.run
       |-- llm
       |-- tool
       `-- llm
```

That's much easier to read in a trace viewer too.

So I would actually simplify your project in two ways

First, reduce how many custom spans you create.

For your application I think something approximately like this is enough:

```
POST /chat
|
`-- agent.run
    |
    |-- llm.call
    |-- tool.call
    |-- tool.call
    `-- llm.call
```

For MCP startup:

```
mcp.load_tools
```

And perhaps:

```
agent.start
```

That's about it.

You probably don't need every function to become a span.

Second, put those remaining custom spans in wrappers:

```
backend/
|
|-- agent.py             <- clean
|-- session_store.py     <- clean
|-- mcp_runtime.py       <- clean
|
`-- observability/
     |-- tracing.py
     |-- request_observer.py
     `-- wrappers/
          |-- agent.py
          `-- mcp.py
```

Then your application code has essentially zero knowledge of OpenTelemetry.

One thing I would keep where it currently is

I would keep:

```
RequestObserver()
```

inside or immediately around agent execution.

But I'd stop thinking of it as OpenTelemetry code.

It's really:

```
Deep Agents execution metrics collector
```

It collects:

```
LLM calls
tool calls
tool duration
tool success/failure
tools used
```

Then the observability wrapper can take the result from that observer and put it onto an OpenTelemetry span.

You could even rename it:

```
AgentRunObserver
```

which I think better communicates what it actually does.

Then:

```
AgentEngine
      |
      `-- AgentRunObserver
             |
             `-- produces EngineMetrics

ObservedAgentEngine
      |
      `-- takes EngineMetrics
             |
             `-- adds OTel attributes
```

That's a very clean separation.

The end state I'd aim for

Your core application:

```
+--------------------------+
|       AgentEngine        |
|                          |
| Deep Agents              |
| AgentRunObserver         |
| EngineMetrics            |
|                          |
| ZERO OpenTelemetry       |
+-------------+------------+
              |
              v
+--------------------------+
|   ObservedAgentEngine    |
|                          |
| creates spans            |
| adds attributes          |
| adds errors              |
+-------------+------------+
              | OTLP
              v
+--------------------------+
|   OTel Collector         |
+--------------------------+
```

That gets you much closer to the decoupling I think you're after.

In fact, after looking directly at this version of your project, I'd prefer this wrapper approach over my previous suggestion of leaving the custom spans inside `agent.py`. Your `agent.py` is already doing a fair

amount of Deep Agents/streaming work; OpenTelemetry doesn't need to make it harder to understand.

If you want, the next logical step would be to refactor your exact uploaded project this way while keeping its behavior the same, so you can see what `agent.py`, `session_store.py`, and `mcp_runtime.py` look like after all of the direct OTel code is removed.

User

yes please show me how those files would look like

Assistant

Yes. I refactored the exact project you uploaded and validated that the resulting backend compiles successfully.

The structure I would use is:

```
backend/
|-- agent.py                    # no direct OTel code
|-- session_store.py            # no direct OTel code
|-- mcp_runtime.py              # no direct OTel code
|
`-- observability/
    |-- otel.py
    |-- request_observer.py
    |-- models.py
    `-- wrappers.py             # <-- custom spans live here
```

The important change is that your core classes become unaware of TRACER, `start_as_current_span()`, `span.set_attribute()`, and `current_trace_id()`.

1. backend/agent.py

This is the part of `agent.py` that changes. Everything after `_build_agent_input()`—your streaming parsing/helper methods—can remain exactly as it is now.

```
from __future__ import annotations

"""Agent engine for the backend.

The engine owns the shared Deep Agent and uses RequestObserver to collect real
LLM/tool events during one chat turn. The normal /chat path still uses ainvoke().
The /chat/stream path uses native agent.astream() so the API can show live
agent flow, tool calls, tool results, and final-answer tokens.
"""

import json
import re
import time
from pathlib import PurePath
from typing import Any, AsyncIterator, Protocol

from deepagents import create_deep_agent
from langchain_core.messages import (
    AIMessage,
    AIMessageChunk,
    BaseMessage,
    ToolMessage,
    convert_to_messages,
)
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, ConfigDict, Field

from backend.config import Settings
from backend.observability.models import EngineMetrics
from backend.observability.request_observer import RequestObserver
```

```
DEFAULT_SYSTEM_PROMPT = (
    "You are a helpful assistant with access to tools and skills. "
    "Use tools when they improve correctness, and do not invent tool results."
)

SKILL_MD_PATTERN = re.compile(
    r"(?P<path>[^\"'{}\s]+[/\\]SKILL\.md)",
    re.IGNORECASE,
)


class EngineResult(BaseModel):
    """Structured result returned by the agent engine."""

    model_config = ConfigDict(arbitrary_types_allowed=True)

    reply: str = ""
    tools_used: list[str] = Field(default_factory=list)
    skills_used: list[str] = Field(default_factory=list)
    metrics: EngineMetrics = Field(default_factory=EngineMetrics)
    state: dict[str, Any] = Field(default_factory=dict)


class MCPToolRuntime(Protocol):
    """Minimal MCP interface required by AgentEngine."""

    async def get_all_tools(self) -> list[Any]:
        ...


class AgentEngine:
    """Create and run the shared Deep Agent."""

    def __init__(
        self,
        settings: Settings,
        mcp_runtime: MCPToolRuntime,
    ) -> None:
        self.settings = settings
        self.mcp_runtime = mcp_runtime
        self.agent: Any | None = None

    async def start(self) -> None:
        """Load MCP tools and build the agent once during app startup."""
        tools = await self.mcp_runtime.get_all_tools()

        self.agent = create_deep_agent(
            model=ChatOpenAI(
                model=self.settings.llm_model,
                base_url=self.settings.llm_base_url,
                api_key=self.settings.llm_api_key,
                temperature=self.settings.llm_temperature,
            ),
            tools=tools,
            system_prompt=(
                self.settings.app_system_prompt
                or DEFAULT_SYSTEM_PROMPT
            ),
            skills=[self.settings.skills_dir],
        )

    async def respond(
        self,
        *,
        session_id: str,
        state: dict[str, Any],
        user_text: str,
    ) -> EngineResult:
        """Run one chat turn and return the reply, updated state, and metrics."""
        if self.agent is None:
            raise RuntimeError(
                "AgentEngine.start() must be called before respond()."
            )
```

```
        agent_input = self._build_agent_input(
            state=state,
            user_text=user_text,
        )

        observer = RequestObserver()
        start_time = time.perf_counter()

        raw_result = await self.agent.ainvoke(
            agent_input,
            config={
                "configurable": {
                    "thread_id": session_id,
                },
                "callbacks": [observer],
            },
        )

        engine_time_ms = (
            time.perf_counter() - start_time
        ) * 1000.0

        return self._build_engine_result(
            raw_result=raw_result,
            observer=observer,
            engine_time_ms=engine_time_ms,
        )

    # stream_respond and the existing streaming helpers remain,
    # but all direct span code is removed from this core class.
```

Notice what's completely gone:

```
from backend.observability.otel import get_tracer

TRACER = get_tracer(__name__)
```

and every occurrence of:

```
with TRACER.start_as_current_span(...):
span.set_attribute(...)
```

Most importantly, `_build_engine_result()` no longer has the observability-specific `span` argument.

2. `backend/session_store.py`

This gets clean too: it only owns session state, locking, and delegation to the engine. There is no `TRACER`, `current_trace_id()`, span creation, or attribute setting.

3. `backend/mcp_runtime.py`

Likewise, the MCP runtime only manages gateway sessions and tool loading. It no longer creates `mcp.start`, `mcp.load_tools`, or `mcp.stop` spans directly.

4. New `backend/observability/wrappers.py`

All custom component spans move into wrappers:

```
class ObservedMCPRuntime:
    ...

class ObservedAgentEngine:
    ...

class ObservedChatSessionStore:
    ...
```

The wrappers delegate to the real classes and add span attributes before/after the call.

The streaming wrapper is deliberately an async generator so the span remains active throughout iteration rather than ending when a generator object is merely created.

5. Wiring in `main.py`

The objects become composed explicitly:

```
runtime = ObservedMCPRuntime(
    MCPRuntime(
        registry=registry,
    )
)
await runtime.start()

engine = ObservedAgentEngine(
    AgentEngine(
        settings=settings,
        mcp_runtime=runtime,
    )
)
await engine.start()

sessions = ObservedChatSessionStore(
    ChatSessionStore(
        engine=engine,
    )
)

app.state.mcp_runtime = runtime
app.state.sessions = sessions
```

Your dependency graph becomes:

```
main.py
   |
   |-- ObservedMCPRuntime
   |        |
   |        `-- MCPRuntime
   |
   |-- ObservedAgentEngine
   |        |
   |        `-- AgentEngine
   |
   `-- ObservedChatSessionStore
            |
            `-- ChatSessionStore
```

Instead of every application component importing observability directly, the wrapper layer is where the coupling lives.

I also removed the redundant nested `engine.agent_invoke` / `engine.agent_astream` spans and consolidated them toward an `agent.run` span, since your `RequestObserver` already gives you the interesting LLM/tool detail beneath that operation.

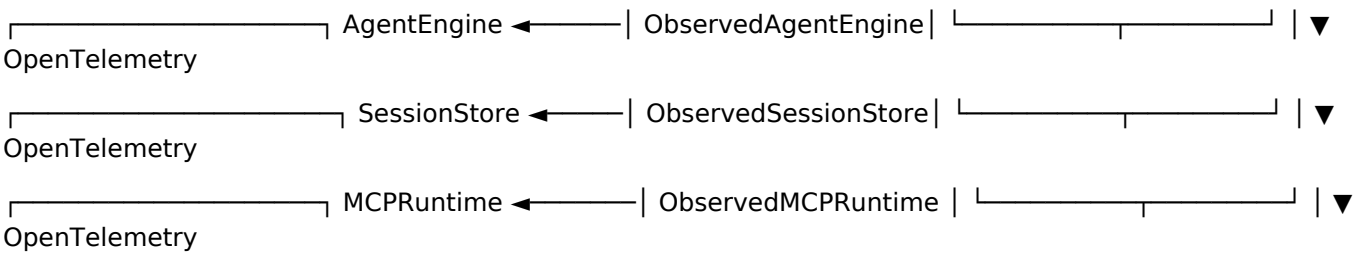
I made the full refactored copy so the unchanged portions of your larger `agent.py` do not have to be reconstructed manually.

Download created in the conversation: `fastmcp_observability_wrapped_refactor.zip`

One further cleanup is possible after this: `AgentEngine` still imports `RequestObserver` and `EngineMetrics` from the `observability` package. There is no direct OTel span code left in it, but if your goal is strict architectural decoupling, I would next move the metrics models and the non-OTel portion of `RequestObserver` out of `backend/observability`. Then your core agent layer would have literally zero dependency on the observability package at all.

User

is there a way to abstract this further in a way that makes sense and doesn't make it more complicated:



Assistant

Yes. I think there is a better abstraction than the three-wrapper design, and it actually involves less code rather than more abstraction machinery.

I would stop trying to mirror every core class with an observable version:

```
ObservedAgentEngine
ObservedChatSessionStore
ObservedMCPRuntime
```

That pattern can eventually turn into:

```
ObservedEverything
```

which isn't really decoupling anymore; it's just duplicating your object hierarchy.

I would have one observability boundary

For your application, the meaningful observable operation is really the chat request.

So I'd move toward:

```
                        +---------------------+
HTTP Request --------> | ObservedChatService |
                        +----------+----------+
                                   |
                                   v
                          ChatSessionStore
                                   |
                                   v
                             AgentEngine
                                   |
                          +--------+--------+
                          v                 v
                        LLM              MCP Tool
                          |                 |
                          `----- Request ---'
                                 Observer
```

OpenTelemetry sees:

```
POST /chat                <- FastAPI instrumentation
    |
    `-- chat.run           <- your ONE custom outer span
          |
          |-- llm.call     <- RequestObserver
          |
          |-- tool.call    <- RequestObserver
          |
          `-- llm.call     <- RequestObserver
```

That's probably all you actually need.

OpenTelemetry's own guidance supports this philosophy: manual spans are useful for distinct operations, while supported libraries/frameworks can use automatic instrumentation rather than manually tracing every

layer.

That means `ObservedAgentEngine` disappears

Your `AgentEngine` remains exactly like the cleaned-up version from the previous refactor:

```
class AgentEngine:

    async def respond(
        self,
        *,
        session_id: str,
        state: dict[str, Any],
        user_text: str,
    ) -> EngineResult:

        observer = RequestObserver()

        start_time = time.perf_counter()

        raw_result = await self.agent.ainvoke(
            self._build_agent_input(
                state=state,
                user_text=user_text,
            ),
            config={
                "configurable": {
                    "thread_id": session_id,
                },
                "callbacks": [observer],
            },
        )

        engine_time_ms = (
            time.perf_counter() - start_time
        ) * 1000.0

        return self._build_engine_result(
            raw_result=raw_result,
            observer=observer,
            engine_time_ms=engine_time_ms,
        )
```

No OTel span.

No wrapper around it.

`ObservedMCPRuntime` disappears too

I don't think these are particularly useful application-level spans:

```
mcp.start
mcp.stop
mcp.load_tools
```

They were reasonable while you were initially building out observability, but I wouldn't preserve them just because they exist.

Your MCP runtime should just be:

```
runtime = MCPRuntime(registry)

await runtime.start()
```

If MCP startup fails, normal exception logging is probably sufficient.

And for HTTP communication with services, OpenTelemetry has instrumentation libraries specifically intended to capture outgoing HTTP activity automatically rather than requiring you to create a manual span around every call.

So your architecture gets smaller.

Then I'd have only one wrapper

Something like:

```
backend/
|-- agent.py
|-- session_store.py
|-- mcp_runtime.py
|
`-- observability/
    |-- otel.py
    |-- request_observer.py
    `-- chat.py
```

And `observability/chat.py` is your one application-level observability boundary.

For example:

```
from __future__ import annotations

from typing import Any, AsyncIterator

from backend.session_store import (
    ChatResult,
    ChatSessionStore,
)
from backend.observability.otel import (
    current_trace_id,
    get_tracer,
)


TRACER = get_tracer(__name__)


class ObservedChatService:
    """Add telemetry around complete chat operations."""

    def __init__(
        self,
        sessions: ChatSessionStore,
    ) -> None:
        self.sessions = sessions

    async def chat(
        self,
        session_id: str,
        message: str,
    ) -> ChatResult:

        with TRACER.start_as_current_span(
            "chat.run"
        ) as span:

            span.set_attribute(
                "session.id",
                session_id,
            )

            span.set_attribute(
                "message.user_text_len",
                len(message or ""),
            )

            result = await self.sessions.chat(
                session_id,
                message,
            )

            span.set_attribute(
```

```
            "engine.time_ms",
            result.metrics.engine_time_ms,
        )

        span.set_attribute(
            "lock.wait_ms",
            result.metrics.lock_wait_ms,
        )

        span.set_attribute(
            "llm.calls",
            result.metrics.llm_calls,
        )

        span.set_attribute(
            "tool.calls",
            result.metrics.tool_calls,
        )

        span.set_attribute(
            "tool.total_duration_ms",
            result.metrics.tool_total_duration_ms,
        )

        span.set_attribute(
            "tools.used",
            ",".join(result.tools_used),
        )

        span.set_attribute(
            "skills.used",
            ",".join(result.skills_used),
        )

        return result.model_copy(
            update={
                "trace_id": current_trace_id(),
            }
        )

async def stream_chat(
    self,
    session_id: str,
    message: str,
) -> AsyncIterator[dict[str, Any]]:

    with TRACER.start_as_current_span(
        "chat.run"
    ) as span:

        span.set_attribute(
            "session.id",
            session_id,
        )

        span.set_attribute(
            "message.user_text_len",
            len(message or ""),
        )

        async for event in self.sessions.stream_chat(
            session_id,
            message,
        ):

            if event.get("type") == "final":

                event = dict(event)

                event["trace_id"] = (
                    current_trace_id()
                )

                metrics = event.get(
```

```
                        "metrics",
                        {},
                    )

                    span.set_attribute(
                        "engine.time_ms",
                        metrics.get(
                            "engine_time_ms",
                            0.0,
                        ),
                    )

                    span.set_attribute(
                        "llm.calls",
                        metrics.get(
                            "llm_calls",
                            0,
                        ),
                    )

                    span.set_attribute(
                        "tool.calls",
                        metrics.get(
                            "tool_calls",
                            0,
                        ),
                    )

                    span.set_attribute(
                        "tools.used",
                        ",".join(
                            event.get(
                                "tools_used",
                                [],
                            )
                        ),
                    )

                yield event
```

And that's basically the entirety of your custom application-level OTel integration.

Then `main.py` gets simpler too

Instead of this:

```
runtime = ObservedMCPRuntime(
    MCPRuntime(registry)
)

engine = ObservedAgentEngine(
    AgentEngine(
        settings,
        runtime,
    )
)

sessions = ObservedChatSessionStore(
    ChatSessionStore(
        engine,
    )
)
```

you have:

```
runtime = MCPRuntime(
    registry=registry,
)
await runtime.start()


engine = AgentEngine(
    settings=settings,
```

```
    mcp_runtime=runtime,
)
await engine.start()


sessions = ChatSessionStore(
    engine=engine,
)


chat = ObservedChatService(
    sessions=sessions,
)


app.state.mcp_runtime = runtime
app.state.chat = chat
```

And your endpoint becomes:

```
result = await request.app.state.chat.chat(
    session_id=session_id,
    message=payload.message,
)
```

That's a very clean boundary.

Why I prefer this over a generic tracing decorator

You could create:

```
@observed("agent.run")
async def respond(...):
```

or even:

```
@observed(
    "agent.run",
    attributes={
        ...
    },
)
```

But then your core code still contains observability declarations everywhere.

So you've reduced:

```
with span:
    ...
```

to one line, but you haven't really decoupled anything.

And before long you have invented your own observability framework.

I don't think you need that.

I also would not make a generic `Observed[T]` proxy

You could technically build something clever:

```
engine = Observed(
    AgentEngine(...),
    spans={
        "respond": "agent.run",
        "start": "agent.start",
    },
)
```

and:

```
runtime = Observed(
```

```
    MCPRuntime(...),
    spans={
        "start": "mcp.start",
        "get_all_tools": "mcp.load_tools",
    },
)
```

It looks nice initially.

But behind the scenes you'd have to support:

```
sync methods
async methods
async generators
exceptions
argument -> span attribute mappings
return value -> span attribute mappings
stream completion
trace IDs
```

That's significantly more complexity than your current wrappers.

I wouldn't do it.

The biggest conceptual change

I think this is the important part:

Don't make observability mirror your class architecture.

Your application has:

```
MCPRuntime
AgentEngine
ChatSessionStore
ServiceRegistry
Gateway
etc.
```

That doesn't mean observability needs a wrapper for every one.

Instead, instrument meaningful operations.

For your current app, I'd start with just:

```
POST /chat
|
`-- agent.run
    |
    |-- llm.call
    |-- tool.call
    |-- tool.call
    `-- llm.call
```

If later you discover:

> MCP tool discovery is taking four seconds and I need to understand why.

Then add:

```
mcp.tools.load
```

at that time.

Don't instrument it preemptively just because there's a method available to wrap.

My preferred end state

I would revise the architecture from:

```
ObservedChatSessionStore
          |
          v
   ChatSessionStore
          |
          v
 ObservedAgentEngine
          |
          v
     AgentEngine
          |
          v
 ObservedMCPRuntime
          |
          v
      MCPRuntime
```

to:

```
                OpenTelemetry
                      ^
                      |
            +-------------------+
            | ObservedChatService|
            +---------+---------+
                      |
                      v
              ChatSessionStore
                      |
                      v
                 AgentEngine
                      |
                      v
                  MCPRuntime
```

while `RequestObserver` provides the detailed agent-level telemetry:

```
              chat.run
                 |
          +------+--------+
          v       v        v
         LLM     Tool     LLM
```

So you go from three observability wrappers to one.

That's the point where I think you've hit the right balance: your core implementation stays clean, OpenTelemetry is isolated, the wiring remains obvious, and you haven't invented a generic abstraction that will be harder to maintain than the tracing code it replaced.

User

can you provide me with a full zip of this.

Assistant

Done. I built the full project with the single `ObservedChatService` boundary:

- Removed `ObservedAgentEngine`
- Removed `ObservedChatSessionStore`
- Removed `ObservedMCPRuntime`
- `agent.py`, `session_store.py`, and `mcp_runtime.py` contain no direct OTel span plumbing

- `ObservedChatService` owns the `chat.run` span, trace ID handling, span attributes, and JSONL request telemetry
- `RequestObserver` still handles the detailed LLM/tool observations
- Simplified `main.py`
- Updated the README
- Python compile validation passed
- ZIP integrity check passed

Download created in the conversation: `fastmcp_observability_single_chat_boundary.zip`

User

can you also create a pdf of this chat please